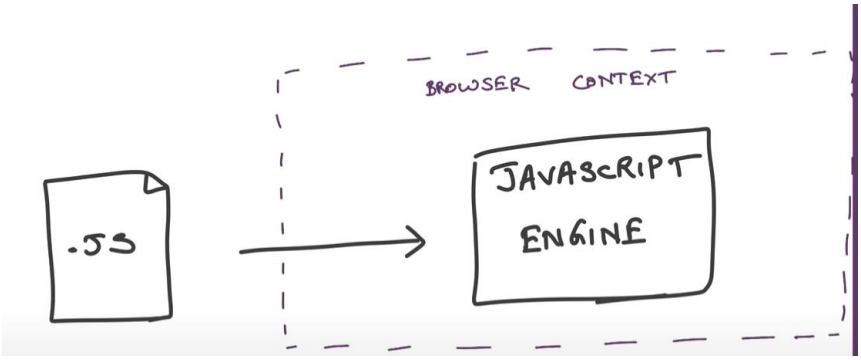
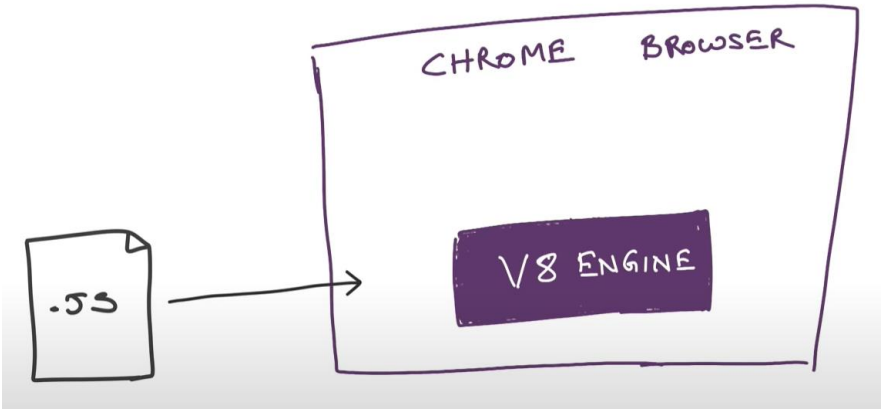
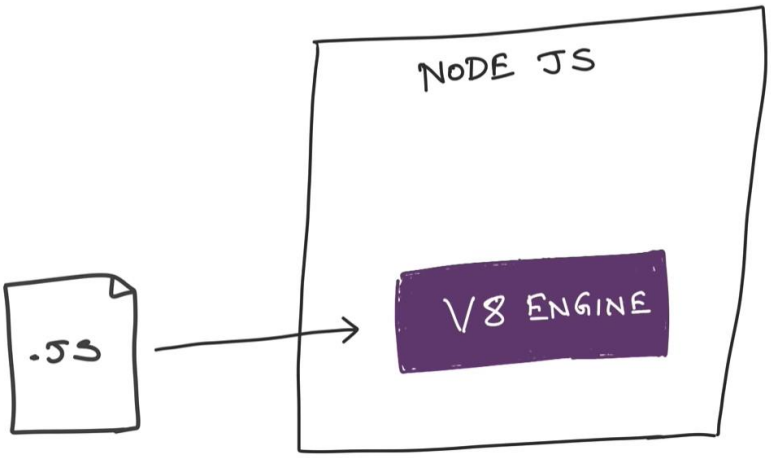
Table of Contents

Javascript Engine Intro	2
1.NodeJs Basics	3
2.NodeJs Complete Tutorial	8
3.Express.js Tutorial	23

Node.js® is a JavaScript runtime built on Chrome's V8 JavaScript engine.







What is Node.js? One Stop Solution for Beginners

Last updated on Apr 29,2020 *2.7K Views*

Swatee Chand
Sr Research Analyst at Edureka. A techno freak who likes to explore...

JavaScript is a prominent language that dominated the market of front-end development since ages. It is a powerful scripting language that helps in developing client-side applications but it failed when there was a necessity to execute applications on the server-side. This is where **Node.js** comes in and provides a better feature set. By the time you finish reading this article, you will be having a complete understanding of what is Node.js and why it is so popular in the market.

Below are the topics covered in this what is Node.js article:

- What is Node.js?
- Node.js Architecture
- Advantages of Node.js
- Applications of Node.js
- Simple Node.js Demo

What is Node.js?

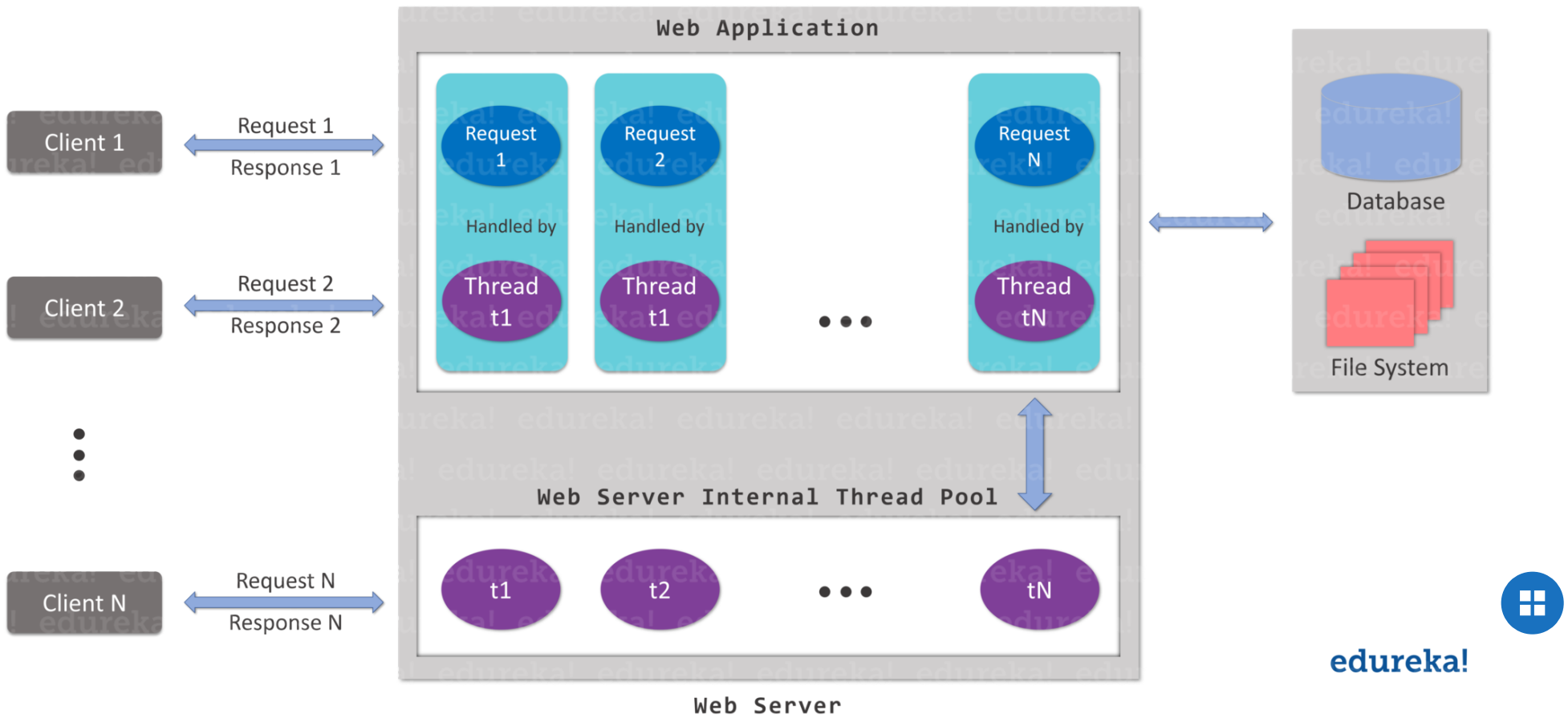
Node.js is a powerful JavaScript framework that is developed on Chrome's V8 JavaScript engine. It helps in compiling JavaScript directly into the native machine code. It is light in weight and is heavily used in the market for developing server-side web applications. Node.js extends JavaScript API which enables the usual server-side functionalities as well. It is generally used for large-scale application development, especially for video streaming sites, single-page application, and other web applications. Node.js makes use of an event-driven, non-blocking I/O model which makes it a right pick for the data-intensive real-time applications.

Like any other programming languages, Node.js makes use of various packages and modules. These are nothing but the libraries containing functions and are imported from npm (node package manager) into our code and utilized in the programs. But in order to understand Node.js in more detail, first, you need to know how it works internally.

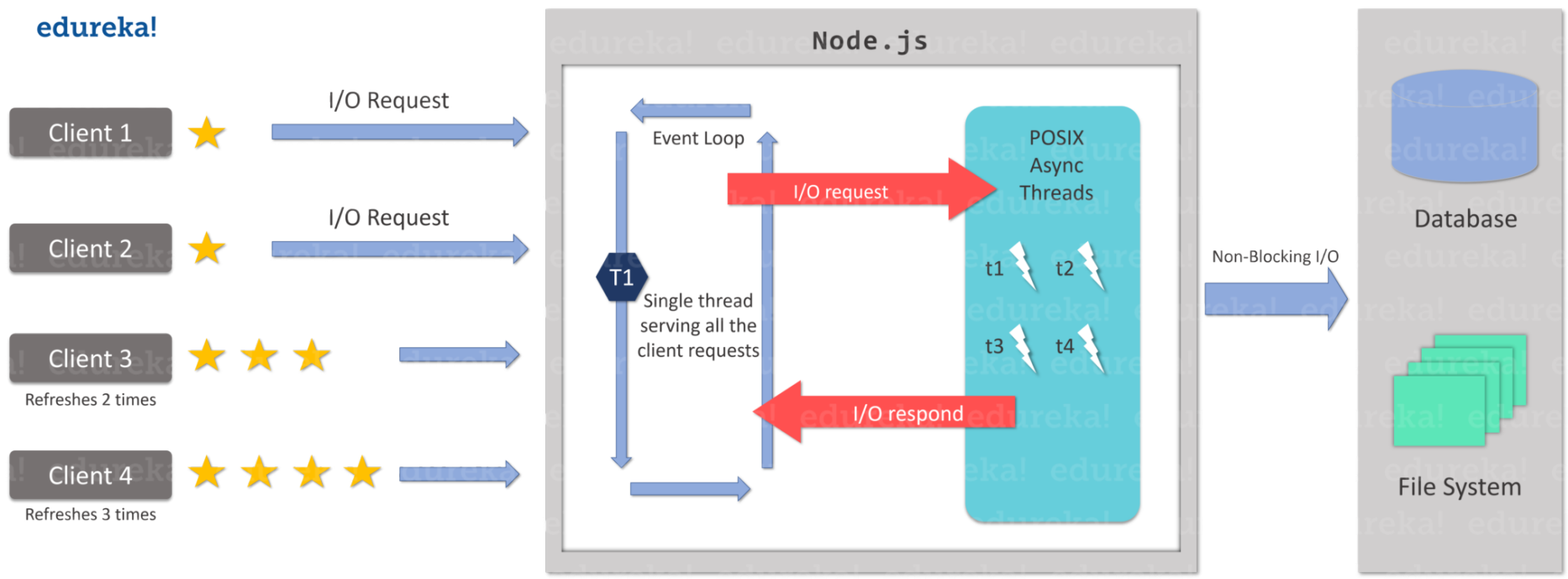
In the next section of this What is Node.js article, I will get into the details of its architecture.

Node.js Architecture

Generally, the server-side technologies like PHP, ASP.NET, Ruby & Java Servers all follow a multi-threaded model. In this traditional architectural approach, each client request creates a new thread or a process.



To avoid this, Node.js uses **Single Threaded Event Loop Model Architecture**. It means that all the client requests on Node.js are executed on the same thread. But this architecture is not just single-threaded, but event-driven as well. It helps Node.js in handling multiple clients concurrently. Below diagram, represents the Single Threaded Event Loop Model architecture.



As you can see in the diagram, the main event loop is single-threaded whereas the I/O workers are executed on distinct threads. This is done to accommodate the entire event loop where most of the Node.js API's are designed to be asynchronous/non-blocking. This reduces the server response time and increases application throughput.

I hope this clears the internal functioning of Node.js. Let now move further in this 'What is Node.js' article and find out various advantages of using Node.js for application development.

Advantages of Node.js

- **Easy to Scale**

  Node.js is mostly preferred by the developers as it is easy to scale the applications developed with Node.js, in horizontal as well as the vertical direction. Also, you can easily assign extra resources to it while scaling the app.

- **Real-Time Web Applications**

  The applications developed in Node.js provides a faster synchronization and because of its single-threaded architecture HTTP overload is also reduced. This is the main reason developers prefer Node.js for developing web-based chat and gaming applications.

- **Faster Suite**

  As you know by now, Node.js runs on the top of Google's V8 engine which by far is the fastest JavaScript engine. Node.js makes use of npm which is an online repository for libraries and tools. Because of npm, the code execution of Node.js application is way faster when compared with other technologies.

- **Easy to Learn**

  Anyone having a brief knowledge about JavaScript can learn to code in Node.js easily as it is a JavaScript framework. The learning curve of Node.js is very shallow and it is less complex to learn and use its various libraries and toolkits.

- **Single Programming Language**

  Node.js can single-handedly build web applications for both, front-end as well as the back-end. Because of this, the deployment of web applications has become a lot easier as most of the web browsers support JavaScript.

- **Caching**

  Node.js enables the functionality of caching of a single module. As a result, you don't have to re-execute the code, each time a request for the first module is generated.

- **Data Streaming**

  Since HTTP requests and responses are considered to be two separate modules in Node.js, they are treated as two separate data streams. Now each time these files are loaded, the overall processing time decreases resulting in better efficiency. Thus, you can stream audios and videos without having to wait for long.

- **Hosting**

  Deployment the Node.js applications has also become very easy with the help of hosting platforms such as PaaS (Platform as a Service) and Heroku.

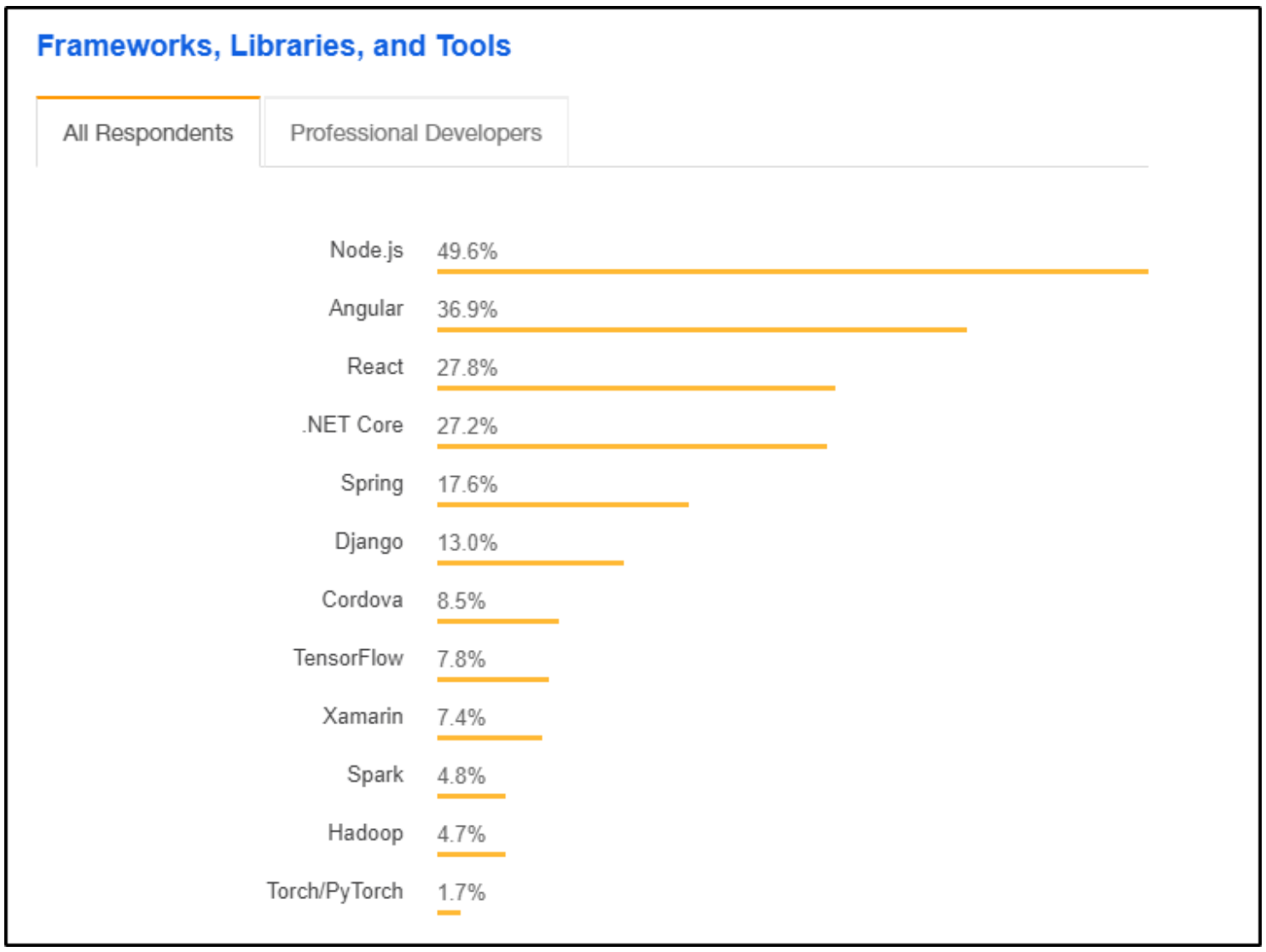
- **Support of Large and Active Community**

  Node.js is backed up by a large and active community of developers. They continuously contribute towards its further development and improvement and come up with innovative solutions.

Let's now move further with this what is Node.js article, and see the various applications of Node.js in today's market.

Applications of Node.js

Node.js tops the popularity chart when compared with other frameworks, libraries, and tools. Below I have added the screenshot of the results held by **StackOverflow**.



Below are the applications which are mostly developed in Node.js:

- Real-Time Chats
- Complex Single-Page Applications
- Real-time collaboration tools
- Live Streaming Applications
- JSON APIs based Applications

Now that you know about the areas of application of Node.js, let's check out the list of the major brands in the market using Node.js in their production:

1. Netflix
2. Linkedin
3. Trello
4. Uber
5. PayPal
6. Medium
7. eBay
8. NASA
9. Groupon
10. Walmart

Now, in order to understand what is Node.js better, let's take a look at its practical implementation.

Simple Demo with Node.js

In order to get your feet wet with Node.js, first, you need to have the Node.js environment installed in your system. To install it, in your system, you can refer to my article on Step by Step Guide to Node.js Installation.

Once done, all you need to do is, open the command prompt and navigate to your project directory. There you need to type in the following command.

```
npm init
```

As soon as you hit enter, you will be asked a few questions related to your project file such as:

```
C:\Users\Swatee_Chand>npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help json` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (swatee_chand)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
```

Once you provide the details and hit 'Enter' your package.json file will be created having details of your project. It should look like below:

```
{
  "name": "nodeintro",
  "version": "1.0.0",
  "description": "What is Node.js Demo",
  "main": "script.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Edureka",
  "license": "ISC"
}
```

Now, the next step is to create the entry point of your application. In order to create this file, type in the below-given code and name the file as you have specified in the json file:

```
const http = require('http');

const hostname = '127.0.0.1';
const port = 8080;

const server = http.createServer((req, res) => {
    res.statusCode = 200;
    res.setHeader('Content-Type', 'text/plain');
    res.end('Welcome to Edureka's Tutorial');
});

server.listen(port, hostname, () => {
    console.log(`Server running at http://${hostname}:${port}/`);
});
```

With this, we come to an end of this article. I hope, now you have a clear understanding of what is Node.js and why you must use it for application development. Now, if this has perked you up for learning Node.js, you can refer to my Node.js Tutorial where I have talked about its fundamental concepts from scratch.

*If you found this "What is Node.js" relevant, check out the **Node.js Certification Training** by Edureka, a trusted online learning company with a network of more than 250,000 satisfied learners spread across the globe.*

Got a question for us? Please mention it in the comments section of this "What is Node.js" article and we will get back to you.

Become a Certified Professional →

Node.js Tutorial – A Complete Guide For Beginners

Last updated on Nov 26,2019 *26.3K Views*

Swatee Chand
Sr Research Analyst at Edureka. A techno freak who likes to explore...

Learn to build a Web Product like Zomato in 60 Days

Attend Edureka Internship Program's Free Demo Session

- Learn how to create a basic version of a Product Homepage from scratch
- Learn more about Edureka's Full-Stack Web Development Internship Program

Enroll Now

If you have ever heard about Node.js then you might know that it is one the most prominent and powerful frameworks of JavaScript. Since its release, it has continued to keep its stranglehold on the IT market. Even with the introduction of new and vibrant JS Frameworks like Angular, React, Meteor etc., the popularity of Node.js never seems to cease. Wondering why? Well, with the help of this Node.js Tutorial, I will be giving you a complete insight into it. So, get ready to fall in love with Node.js.

In this Node.js Tutorial, I will be discussing the below topics:

- What is Node.js?
- NPM (Node Package Manager)
- Node.js Modules
- JSON File
- Node.js Basics
- File System
- Events
- HTTP Module
- Express.js
- Node.js Tutorial Step By Step Application Development with Express.js

What is Node.js?

Node.js is a powerful framework developed on **Chrome's V8 JavaScript engine** that compiles the JavaScript directly into the native machine code. It is a lightweight framework used for creating server-side web applications and extends JavaScript API to offer usual server-side functionalities. It is generally used for large-scale application development, especially for video streaming sites, single page application, and other web applications. Node.js *makes* use of an event-driven, non-blocking I/O model which makes it a right pick for the data-intensive real-time applications.

Like any other programming languages, node.js makes use of packages and modules. These are the libraries that contain various functions and are imported from npm (node package manager) into our code and utilized in the programs. Some of the major features, that define Node.js are listed below:

Features of Node.js

1. **Open Source**
   Node.js is an open source framework MIT license that is supported by a huge community. Its community is pretty much active have contributed to add new capabilities to Node.js applications.
2. **Simple and Fast**
   Since Node.js is built on Google Chrome's V8 JavaScript Engine, its libraries are capable of fast code execution.
3. **Asynchronous**
   All the libraries of Node.js are asynchronous which means the Node.js based servers never wait for an API to send back the response and move on to the next API.
4. **High Scalability**
   Because of the event mechanism, Node.js is highly scalable and aids the server in a non-blocking response.
5. **Single-Threaded**
   With the help of event looping, Node.js is able to follow the single-threaded model. This lets a single program to handle multiple requests.

Become a Certified Professional→

Node.js can be easily built and deployed on various platforms like Windows, MAC, and Linux.

Let's now advance further and see how to deploy the actual code on the browser. But before that, you need to download and install in your systems. You can refer my other article to know the complete **Node.js installation process**.

So now, let's move further in this Node.js Tutorial, where I will talk about the most important component of Node.js i.e., npm.

NPM (Node Package Manager)

NPM stands for Node Package Manager which as the name suggests is a package manager for Node.js packages/modules. From Node version 0.6.0. onwards, npm has been added as default in the node installation. It saves you from the hassle of installing npm explicitly.

NPM basically helps in two ways:

1. Provides and hosts Online repositories for node.js packages/modules which can be easily downloaded and used in our projects. You can find them here: npmjs.com
2. Provides the Command line utility in order to install various Node.js packages, manage Node.js versions and dependencies of the packages.

But now, you must be wondering what exactly these modules are and how do they help us in building the Node.js applications. Well, in the next section of this Node.js tutorial, I will give you a complete insight into Node.js modules.

Node.js Modules

The modules in Node.js represents various functionalities that are bundled up into single or multiple JS files. These modules have a unique context, thus, they never interfere nor pollute the scope of other modules.

These modules enable the code reusability and enhance the ease of usage. Node.js basically provides three types of modules:

1. Core Modules
2. Local Modules
3. Third-Party Modules

Core Module

Since Node.js is a very ***lightweight*** framework, the core modules bundle the absolute minimum functionalities. These modules generally get loaded when the Node process starts its execution. All you need to do is, import these core modules in order to use them in your code.

Below I have listed down a few of the important Core modules.

Core Module	Description
http	Contains classes, methods, and events required to create Node.js HTTP server
url	Contains methods for URL resolution and parsing in Node
querystring	Contains methods to deal with a query string of Node
path	Contains methods to deal with file paths
fs	Contains classes, methods, and events to work with file I/O
util	Contains utility functions that can be useful for programmers

You can load your core module, using the below code:

```
var module = require('module_name');
```

Lets now see, what are 'local modules'.

Local Modules

The local modules of Node.js are custom modules that are created locally by user/developer in the application. These modules can include various functionalities bundled into distinct files and folders which can be easily distributed in the Node.js community using NPM.

These modules are loaded in a similar way to core modules. Let show you, how to do it using a basic example.

Become a Certified Professional→

```
    console.log('Name: ' + name);
  },
  domain:function (domain) {
    console.log('Domain: ' + domain);
  }
};

module.exports = detail;
```

Include your module file in your main application file.

```
var myLogModule = require('./Local_module.js');
myLogModule.name('Edureka');
myLogModule.domain('Education');
```

Now you can execute these files using below command:

```
node application.js
```

Let me now show you what are external modules.

External Modules

You can use the external or 3rd party modules only by downloading them via NPM. These modules are generally developed by other developers and are free to use. Few of the best external modules are express, react, gulp, mongoose, mocha etc.

Globally Loading the 3rd party modules:

Node.js Certification Training

Instructor-led Sessions

Assignments

Lifetime Access

24 x 7 Expert Support

Explore Curriculum

```
npm install --g <module_name>
```

Include your module file in your main application file:

```
npm install --save <module_name>
```

JSON File

The **package.json file** in Node.js is the heart of the entire application. It is basically the manifest file that contains the metadata of the project. Thus, understanding and working with this file becomes very important for a successful Node project development.

The package.json file generally consists of the metadata of the application, which is further categorized into below two categories:

1. **Identifying metadata properties:** This consists of properties like the project name, current module version, license, author of the project, project description etc.
2. **Writing directly to file:** You can directly write the necessary information into the package.json file and include it, in your project.

By now you have already familiarized with the various components of Node JS application. In the next section of this Node.js Tutorial, I will be sharing some Node Js basics so that we can start off with the hands on.

Node.js Basics

Become a Certified Professional→

Data Types

Like any other programming language, Node.js has various datatypes, which are further categorized into Primitive and Non-Primitive datatypes.

Primitive Data Types are:

1. String
2. Number
3. Boolean
4. Null
5. Undefined

Non-Primitive Data Types are:

1. Object
2. Date
3. Array

Variables

Variable are entities that hold values which may vary during the course of a program. To create a variable in Node.js, you need to make use of a reserved keyword var. You do not have to assign a data type, as the compiler will automatically pick it.

Syntax:

```
var varName = value;
```

Operators

Node.js supports the below operators:

Operator Type	Operators
Arithmetic	+, -, /, *, %, ++, —
Assignment	=, +=, -=, *=, /=, %=
Conditional	=?
Comparison	==, ===, !=, !==, >, >=, <, <=,
Logical	&&, ||, !
Bitwise	&, |, ^, ~, <<, >>, >>>

Functions

Functions in Node.js is a block of code that has a name and is written to achieve a particular task. You need to use the keyword function to create it. A function is generally a two-step process. First is defining the function and the second is invoking it. Below is the syntax of creating and invoking a function:

Example:

```
//Defining a function
function display_Name(firstName, lastName) {
  alert("Hello " + firstName + " " + lastName);
}

//Invoking the function
display_Name("Park", "Jimin");
```

Objects

An object is a non-primitive data type that can hold multiple values in terms of properties and methods. Node.js objects are standalone entities as there is no concept of class. You can create an object in two ways:

1. Using Object literal
2. Using Object constructor

Example:

Become a Certified Professional→

```
  lastName: 'Choi',
  age: 35,
  salary:50000,
  //method
  getFullName: function () {
    return this.firstName + ' ' + this.lastName
  }
};
```

File System

To access the physical file system, Node.js makes use of the **fs** module which basically takes care of all asynchronous and synchronous file I/O operations. This module is imported using the below command:

```
var fs = require('fs');
```

Some of the general use for the File System modules are:

- **Read files**
    1. fs.readFile()

```
var http = require('http');
var fs = require('fs');
http.createServer(function (req, res) {
  fs.readFile('script.txt', function(err, data) {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write(data);
    res.end();
  });
}).listen(8080);
```

- **Create files**
    1. appendFile()
    2. open()
    3. writeFile()
- **Update files**
    1. fs.appendFile()
    2. fs.writeFile()
- **Delete files**
    1. fs.unlink()
- **Rename files**
    1. fs.rename()

So, with these commands, you can pretty much perform all the required operations on your files. Let's now move further with this Node.js Tutorial and see what are Events and how they are handled in Node.js.

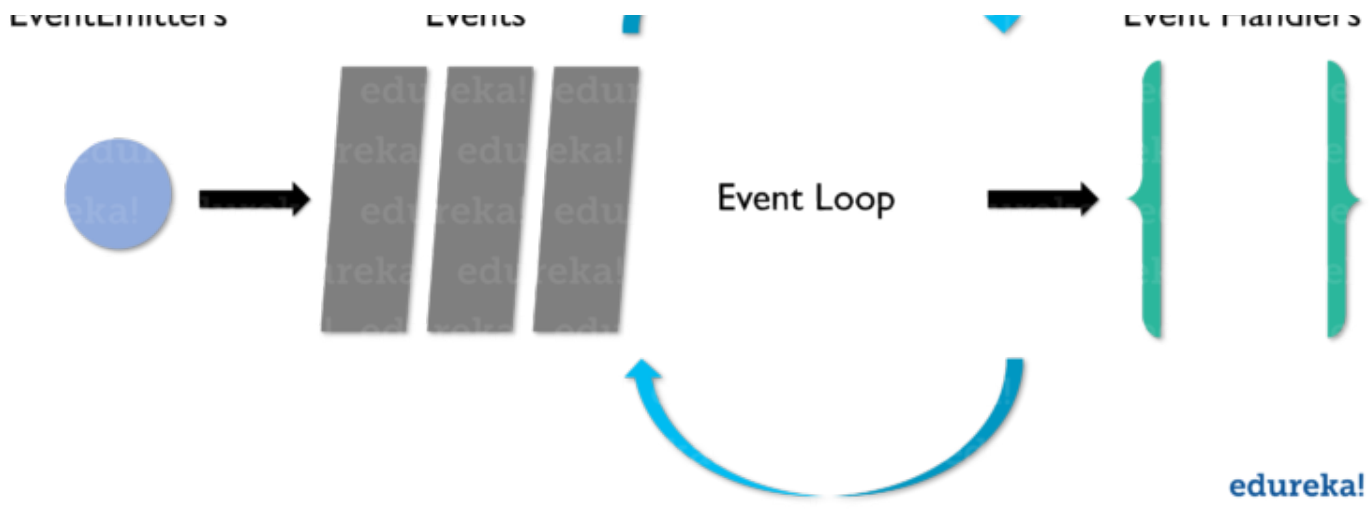
Events

As I have already mentioned, Node.js applications are single threaded and event-driven. Node.js supports concurrency as it is event-driven, and thus makes use of concepts like events and callbacks. The async function calls help Node.js in maintaining concurrency throughout the application.

Basically, in a Node.js application, there is a main loop which waits and listens for events, and once any event is completed, it immediately initiates a callback function.

Below diagram represents how the events are driven in Node.js.

Become a Certified Professional→



One thing that you must note here is that, even though events look similar to callback functions but the difference lies in their functionalities. When an asynchronous function returns its results callbacks are invoked on the other hand event handling completely works on the observer pattern. And in Node.js, methods which listen to the events are called the observers. The moment, an event is triggered, its listener function automatically starts executing. Event modules and EventEmitter class provide multiple in-built events which are used to bind events with event listeners. Below I have written down the syntax for that.

ogramming & Frameworks Training

FULL STACK WEB DEVELOPMENT INTERNSHIP PROGRAM

Full Stack Web Development Internship Program
Reviews
5(2011)

JAVA CERTIFICATION TRAINING COURSE

Java Certification Training Course
Reviews
4(47942)

PYTHON SCRIPTING CERTIFICATION TRAINING

Python Scripting Certification Training
Reviews
5(10226)

PYTHON DJ TRAINING A CERTIFICATI

Python Django Tra and Certification
Reviews
5(5754)

Binding Event to an Event Listener

```
// Import events module
var my_Events = require('events');
// Create an eventEmitter object
var my_EveEmitter = new my_Events.EventEmitter();
```

Binding Event Handler to an Event

```
// Binding event and event handler
my_EveEmitter.on('eventName', eventHandler);
```

Firing an Event

```
// Fire an event
my_EveEmitter.emit('eventName');
```

Now let's try to implement the things that I have discussed in this Node.js Event section. The below code shows a simple representation of Event execution in Node.js.

Become a Certified Professional→

```
        for (var i = 1; i <= num; i++) {
          emt.emit('BeforeProcess', i);
          console.log('Processing Iteration:' + i);
          emt.emit('AfterProcess', i);
        }
      }
      , 5000)
  return emt;
}
var it = iterateProcessor(5);

it.on('BeforeProcess', function (info) {
  console.log('Starting the process for ' + info);
});

it.on('AfterProcess', function (info) {
  console.log('Finishing processing for ' + info);
```

In the next section of this Node.js Tutorial, I will give you insights on one of the most important module of Node.js called the HTTP Module.

HTTP Module

Generally, Node.js is used for developing server-based applications. But using the module, you can easily create web servers that can respond to the client requests. Thus it is also referred to Web Module and provides modules like HTTP and request that facilitate Node.js in processing the server requests.

You can easily include this module in your Node.js application just by writing the below code:

```
var http = require('http');
```

Below I have written a code, that shows how a Web Server is developed in Node.js.

```
//calling http library
var http = require('http');
var url = require('url');

//creating server
var server = http.createServer(function (req, res) {
  //setting content header
  res.writeHead(200, ('Content-Type', 'text/html'));
  var q = url.parse(req.url, true).query;
  var txt = q.year + " " + q.month;
  //send string to response
  res.end(txt);
});

//assigning 8082 as server listening port
server.listen(8082);
```

In the next section of this Node.js Tutorial, I will be talking about Express.js which is heavily used for server-side web application development.

Express.js

Express.js is a framework built on top of Node.js that facilitates the management of the flow of data between server and routes in the server-side applications. It is a lightweight and flexible framework that provides a wide range of features required for the web as well as mobile application development.

Express.js is developed on the middleware module of Node.js called ***connect***. The connect module further makes use of **http** module to communicate with Node.js. Thus, if you are working with any of the connect based middleware modules, then you can easily integrate with Express.js.

Not, only this, few of the major advantages of Express.js are:

- Makes web application development faster
- Helps in building mobile and web application of single-page, multi-page, and hybrid types
- Express provides two templating engines namely, Jade and EJS
- Express follows the Model-View-Controller (MVC) architecture
- Makes integration with databases such as MongoDB, Redis, MySQL
- Defines an error handling middleware
- Simplifies configuration and customization easy for the application.

Become a Certified Professional→

Lets now see a simple example to understand, how Express.js works with Node.js to ease our work. But before you start working with Express.js, you need to install it in your system.

To install Express.js globally you can use the below command:

```
npm install -g express
```

Or, if you want to install it locally into your project folder, you need to execute the below command:

```
npm install express --save
```

Since we are done with all the preparations, let's now jump directly into practical implementation. Here, I will be showing a simple User Authentication Application using Node.js and Express.js.

Node.js Tutorial Step By Step Application Development with Express.js

For this, we will be needing below files:

- package.json
- script.js
- views
  - index.jade
  - login.jade
  - secure.jade
  - unauthorized.jade
  - welcome.jade
- lib
  - routes.js

So, let's start with **package.json**.

```
{
  "author": "Edureka",
  "name": "Express_Demo",
  "description": "Express with Node.js",
  "version": "0.0.0",
  "scripts": {
    "start": "node script.js"
  },
  "engines": {
    "node": "~0.4.12"
  },
  "dependencies": {
    "connect-flash": "^0.1.1",
    "cookie-parser": "^1.4.3",
    "express": "^3.21.2",
    "jade": "^0.20.3",
    "req-flash": "0.0.3"
  },
  "devDependencies": {}
}
```

Next, you need to create the **script.js**.

Become a Certified Professional→

```
var app = express();
const flash = require('connect-flash');
var cookieParser = require('cookie-parser')
var server = http.createServer(app);

function checkAuth (req, res, next) {
   console.log('checkAuth ' + req.url);

   // don't serve /secure to those not logged in
   if (req.url === '/secure' && (!req.session || !req.session.authenticated)) {
      res.render('unauthorised', { status: 403 });
      return;
   }

   next();
}
   app.use(flash());
   app.use(cookieParser());
   app.use(express.session({ secret: 'example' }));
   app.use(express.bodyParser());
   app.use(checkAuth);
   app.use(app.router);
   app.set('view engine', 'jade');
   app.set('view options', { layout: false });

require('./lib/routes.js')(app);

app.listen(port);
console.log('Node listening on port %s', port);
```

Now, create a folder named view, under which you will be adding the jade files that will be responsible for various page views. The first view file you need to create is **index.jade**.

```
!!! 5
html(lang='en')
   head
      title User Authentication Example
      <link href="https://stackpath.bootstrapcdn.com/bootswatch/4.2.1/darkly/bootstrap.min.css"
rel="stylesheet" integrity="sha384-F7cPHYSdE3FzBKvdxDKyFow9Y28Yq7KGHQOKpOI+rwc8s6jJnp///mpEJAib2EM6"
crossorigin="anonymous">

   body
      h1
         center Authentication Demo using Express
         h3 Navigate to
         h4
         ul
            li: a(href="/secure") Secure content
            li: a(href="/welcome") Welcome page
            li: a(href="/logout") Logout
```

Now, create the **login.jade** file.

Become a Certified Professional→

```
      title Express authentication example
      <link href="https://stackpath.bootstrapcdn.com/bootswatch/4.2.1/darkly/bootstrap.min.css"
rel="stylesheet" integrity="sha384-F7cPHYSdE3FzBKvdxDKyFow9Y28Yq7KGHQOKpOI+rwc8s6jJnp///mpEJAib2EM6"
crossorigin="anonymous">
   body
      h1
         center Sign-in to this Express authentication example
      center
         p Use <i>user</i> for the username and <i>pass</i> for the password.

         form(method='post')
            p
            label(for='username') Email Address
            input(type='text', name='username', class='form-control', id='exampleInputPassword1' ,
placeholder='Email', style='width:400px;')
            p
            center
            label(for='password') Password
            input(type='password', name='password', class='form-control' , id='exampleInputPassword1',
placeholder='Password', style='width:400px;')

            p
            center <button type="submit" class="btn btn-info">Submit</button>
      - each message in flash
         h4(style="color: red;") #{message}
```

Next step is to create the **welcome.jade**.

Node.js Certification Training

Weekday / Weekend Batches

See Batch Details

```
!!! 5
html(lang='en')
   head
      title User Authentication Example
      <link href="https://stackpath.bootstrapcdn.com/bootswatch/4.2.1/darkly/bootstrap.min.css"
rel="stylesheet" integrity="sha384-F7cPHYSdE3FzBKvdxDKyFow9Y28Yq7KGHQOKpOI+rwc8s6jJnp///mpEJAib2EM6"
crossorigin="anonymous">
   body
      h1
         center Welcome To The Edureka Tutorial!
```

Next, create the **secure.jade** file.

Become a Certified Professional→

```
      title Express Authentication Example
      <link href="https://stackpath.bootstrapcdn.com/bootswatch/4.2.1/darkly/bootstrap.min.css"
rel="stylesheet" integrity="sha384-F7cPHYSdE3FzBKvdxDKyFow9Y28Yq7KGHQOKpOI+rwc8s6jJnp///mpEJAib2EM6"
crossorigin="anonymous">
   body
      h1
         center Hi, secure user.
      p Navigate to
         ul
            li: a(href="/secure") Secure content
            li: a(href="/welcome") Welcome page
            li: a(href="/logout") Logout
```

Now, create the **unauthorized.jade** file.

```
!!! 5
html(lang='en')
   head
      title User Authentication Example
      <link href="https://stackpath.bootstrapcdn.com/bootswatch/4.2.1/darkly/bootstrap.min.css"
rel="stylesheet" integrity="sha384-F7cPHYSdE3FzBKvdxDKyFow9Y28Yq7KGHQOKpOI+rwc8s6jJnp///mpEJAib2EM6"
crossorigin="anonymous">
   body
      h1
         center Unauthorized
      p You're unauthorized to view this page.
      p Please <a href="/login">login</a> to continue
```

Now, you need to create a folder and name it **lib**. Now, create a **route.js** file which will map all the pages.

Become a Certified Professional→

```
   res.render('index');
});

app.get('/welcome', function (req, res, next) {
   res.render('welcome');
});

app.get('/secure', function (req, res, next) {
   res.render('secure');
});

app.get('/login', function (req, res, next) {
   res.render('login', {flash: req.flash() } );
});

app.post('/login', function (req, res, next) {

   // you might like to do a database look-up or something more scalable here
   if (req.body.username && req.body.username === 'user' && req.body.password && req.body.password ===
'pass') {
      req.session.authenticated = true;
      res.redirect('/secure');
   } else {
      req.flash('error', 'Username and password are incorrect');
      res.redirect('/login');
   }
});

app.get('/logout', function (req, res, next) {
   delete req.session.authenticated;
   res.redirect('/');
});

};
```

Now if you want to execute this code on your own then you can download it from here: ***Node.js Tutorial PDF***.

With this, we come to an end of this Node.js Tutorial. I hope I was able to explain the concepts of Node.js from the ground up.

*If you found this "Node.js Tutorial" relevant, check out the **Node.js Certification Training** by Edureka, a trusted online learning company with a network of more than 250,000 satisfied learners spread across the globe.*

Got a question for us? Please mention it in the comments section of this Node.js Tutorial and we will get back to you.

Recommended videos for you

Introduction to JAVA/J2EE & SOA

Watch Now

Microsoft Sharepoint 2013 : The Ultimate Enterprise Collaboration Platform

Watch Now

Responsive WEB APP using cakePHP

Watch Now

Node JS Express: Ste Create Restful Web

Watch Now

Become a Certified Professional →

Methods and Method Overloading in Java

Read Article

Everything You Need To Know About HTML Textarea

Read Article

Node JS Installation – Know How To Download & Install Node.js

Read Article

What is System Clas and how to impleme

Read Article

‹›

Comments

1 Comment

csps1343 says:
Mar 29, 2019 at 5:39 am GMT

Great stuff, Caleb. So many awesome post ideas. We at CSPS Protocol also provides node js tutorial and training services. Thanks

Reply

Join the discussion

Trending Courses in Programming & Frameworks

Full Stack Web Development Internship Program

- 3k Enrolled Learners
- Weekend/Weekday
- Live Class

Reviews

★★★★★ 5 (850)

Java Certification Training Course

- 48k Enrolled Learners
- Weekend
- Live Class

Reviews

★★★★☆ 4 (19200)

Python Scripting Certification Training

- 11k Enrolled Learners
- Weekend
- Self Paced

Reviews

★★★★★ 5 (4100)

Python Django Tr Certification

- 6k Enrolled Learner
- Weekend
- Live Class

Reviews

★★★★★ 5 (2350

Become a Certified Professional →

Artificial Intelligence | BI and Visualization | Big Data | Blockchain | Cloud Computing | Cyber Security | Data Science | Data Warehousing and ETL | Databases | DevOps | Digital Marketing | Enterprise | Front End Web Development | Mobile Development | Operating Systems | Project Management and Methodologies | Robotic Process Automation | Software Testing | Systems & Architecture

edureka!

TRENDING CERTIFICATION COURSES

DevOps Certification Training
AWS Architect Certification Training
Big Data Hadoop Certification Training
Tableau Training & Certification
Python Certification Training for Data Science
Selenium Certification Training
PMP® Certification Exam Training
Robotic Process Automation Training using UiPath
Apache Spark and Scala Certification Training
Microsoft Power BI Training
Online Java Course and Training
Python Certification Course

TRENDING MASTERS COURSES

Data Scientist Masters Program
DevOps Engineer Masters Program
Cloud Architect Masters Program
Big Data Architect Masters Program
Machine Learning Engineer Masters Program
Full Stack Web Developer Masters Program
Business Intelligence Masters Program
Data Analyst Masters Program
Test Automation Engineer Masters Program
Post-Graduate Program in Artificial Intelligence & Machine Learning
Post-Graduate Program in Big Data Engineering

COMPANY

About us
News & Media
Reviews
Contact us
Blog
Community
Sitemap
Blog Sitemap
Community Sitemap
Webinars

WORK WITH US

Careers
Become an Instructor
Become an Affiliate
Become a Partner
Hire from Edureka

DOWNLOAD APP

CATEGORIES

CATEGORIES

Cloud Computing | DevOps | Big Data | Data Science | BI and Visualization | Programming & Frameworks | Software Testing | Project Management and Methodologies | Robotic Process Automation | Frontend Development | Data Warehousing and ETL | Artificial Intelligence | Blockchain | Databases | Cyber Security | Mobile Development | Operating Systems | Architecture & Design Patterns | Digital Marketing

TRENDING BLOG ARTICLES

TRENDING BLOG ARTICLES

Selenium tutorial | Selenium interview questions | Java tutorial | What is HTML | Java interview questions | PHP tutorial | JavaScript interview questions | Spring tutorial | PHP interview questions | Inheritance in Java | Polymorphism in Java | Spring interview questions | Pointers in C | Linux commands | Android tutorial | JavaScript tutorial | jQuery tutorial | SQL interview questions | MySQL tutorial | Machine learning tutorial | Python tutorial | What is machine learning | Ethical hacking tutorial | SQL injection | AWS certification career opportunities | AWS tutorial | What Is cloud computing | What is blockchain | Hadoop tutorial | What is artificial intelligence | Node Tutorial | Collections in Java | Exception handling in java | Python Programming Language | Python interview questions | Multithreading in Java | ReactJS Tutorial | Data Science vs Big Data vs Data Analyt... | Software Testing Interview Questions | R Tutorial | Java Programs | JavaScript Reserved Words and Keywor... | Implement thread.yield() in Java: Exam... | Implement Optical Character Recogniti... | All you Need to Know About Implemen...

Become a Certified Professional →

"PMP®","PMI®", "PMI-ACP®" and "PMBOK®" are registered marks of the Project Management Institute, Inc. MongoDB®, Mongo and the leaf logo are the registered trademarks of MongoDB, Inc.

Express.js Tutorial – One Stop Solution for Beginners

Last updated on Apr 29,2020 *4.8K Views*

Express.js is a fast and lightweight framework used majorly for web application development and the Node.js Developers all over the world are totally in love with this framework. Express.js provides all the features of web application without overshadowing the Node.js features. Through the medium of this Express.js Tutorial, I will be giving you a complete insight into the fundamentals of Express.js which will help you get started with it.
Below are the topics covered in this Express.js Tutorial:

- What is Express.js?
- Features of Express.js
- Express.js Installation
- Express.js Fundamental Concepts
  - Routing and HTTP Methods
  - Middleware
  - Cookies
  - REST API
  - Scaffolding
  - Database Connectivity
  - Templating
  - Error Handling

Let's get started with this Express.js Tutorial for beginners.

What is Express.js?

In layman terms, Express.js is a web application framework that is built on top of Node.js. It provides a minimal interface with all the tools required to build a web application. Express.js adds flexibility to an application with a huge range of modules available on npm that you can directly plug into Express as per requirement. It helps in easy management of the flow of data between server and routes in the server-side applications. It was developed by **TJ Holowaychuk** and was released in the market on 22nd of May, 2010. Formerly it was managed by IBM but currently, it is placed under the stewardship of the Node.js Foundation incubator.

Express is majorly responsible for handling the backend part in the MEAN stack. Mean Stack is the open-source JavaScript software stack that is heavily used for building dynamic websites and web applications in the market. Here, **MEAN** stands for **M**ongoDB, **E**xpress.js, **A**ngularJS, and **N**ode.js.

That being said, let's now move further with this Express.js Tutorial and find out the various features of this framework.

Features of Express.js

Below I have picked out a few of the most important features of Express.js:

1. Express quickens the development pace of a web application.
2. It also helps in creating mobile and web application of single-page, multi-page, and hybrid types
3. Express can work with various templating engines such as Pug, Mustache, and EJS.
4. Express follows the Model-View-Controller (MVC) architecture.
5. It makes the integration process with databases such as MongoDB, Redis, MySQL effortless.
6. Express also defines an error-handling middleware.
7. It helps in simplifying the configuration and customization steps for the application.

These were a few of the major advantages of using Express.js in web application development. In the next section of this Express.js Tutorial article, let's me show you how to install Express.js in your system.

Installing Express.js

In order to install Express.js in your system, first, you need to make sure that you have Node.js already installed. If not, then don't worry, just refer to my article on Node.js Installation which will guide you in a step by step manner. Once you are done with Node.js installation, the next step is to install Express.

To install Express.js, first, you need to create a project directory and create a package.json file which will be holding the project dependencies. Below is the code to perform the same:

```
npm init
```

Now, you can install the express.js package in your system. To install it globally, you can use the below command:

```
npm install -g express
```

Or, if you want to install it locally into your project folder, you need to execute the below command:

```
npm install express --save
```

Now that you are already done with the installation process, let's now get our feet wet with Express.js. In the next section of this Express.js Tutorial, I will be walking you through the fundamental concepts of the Express framework.

Express.js Fundamental Concepts

So, the first topic I will be covering under Express.js fundamentals is Routing.

1. Routing and HTTP Methods

Routing refers to the process of determining a specific behavior of an application. It is used for defining how an application should respond to a client request to a particular route, path or URI along with a particular HTTP Request method. Each route can contain more than one handler functions, which is executed when the user browses for a specific route. Below is the structure of Routing in Express:

```
app.METHOD(PATH, HANDLER)
```

Where:

- app is just an instance of Express.js. You can use any variable of your choice.
- METHOD is an HTTP request method such as get, set, put, delete, etc.
- PATH is the route to the server for a specific webpage
- HANDLER is the callback function that is executed when the matching route is found.

There are four main HTTP methods that can be supplied within the request. These methods help in specifying the operation requested by the user. Below table lists down all methods along with their explanations:

Method	Description
1. GET	The HTTP GET method helps in requesting for the representation of a specific resource by the client. The requests having GET just retrieves data and without causing any effect.
2. POST	The HTTP POST method helps in requesting the server to accept the data that is enclosed within the request as a new object of the resource as identified by the URI.
3. PUT	The HTTP PUT method helps in requesting the server to accept the data that is enclosed within the request as an alteration to the existing object which is identified by the provided URI.
4. DELETE	The HTTP DELETE method helps in requesting the server to delete a specific resource from the destination.

Below is an example showing usage of all the HTTP methods.

```
var express = require('express');
const app = express();
app.use(express.json());

app.get('/', function (req, res) {
    console.log("GET Request Received");
    res.send('
<h2 style= "font-family: Malgun Gothic; color: blue; "> Welcome to Edureka's Express.js Tutorial!/h2>');
})

app.post('/addcourse', function (req, res) {
    console.log("POST Request Received");
    res.send('
<h2 style="font-family: Malgun Gothic; color: green; ">A course new Course is Added!</h2>

');
})
app.delete('/delete', function (req, res) {
    console.log("DELETE Request Received");
    res.send('
<h2 style="font-family: Malgun Gothic; color: darkred;">A Course has been Deleted!!</h2>

 ');
})
app.get('/course', function (req, res) {
    console.log("GET Request received for /course URI");
    res.send('
<h2 style="font-family: Malgun Gothic; color:blue;">This is an Available Course!</h2>

');
})

//PORT ENVIRONMENT VARIABLE
const port = process.env.PORT || 8080;
app.listen(port, () => console.log(`Listening on port ${port}..`));
```

In the next section of this Express.js Tutorial, I will be introducing you to Middlewares.

2. Middleware

In express, middleware functions are the functions which have access to the request and response objects along with the next function present in the application's request-response cycle. These functions are capable of performing the below-listed tasks:

- Execution of any code
- Modify the request and the response objects.
- End applications request-response cycle.
- Call the next middleware present in the cycle.

Note that, in case, the current function doesn't terminate the request-response cycle then it must invoke the next() function in order to pass on the control to the next available middleware function. If not done, then the request will be left incomplete. Below I have listed down the majorly used middlewares in any Express.js application:

- Application-level middleware
- Router-level middleware
- Error-handling middleware
- Built-in middleware
- Third-party middleware

Now, in order to understand about the middleware functions in detail, let me show you a small example of the same:

```
var express = require('express')
var app = express()

var requestDate = function (req, res, next) {
    req.requestDate = Date.now()
    next()
}

app.use(requestDate)

app.get('/', function (req, res) {
    var responseMsg = '
<h2 style="font-family: Verdana; color: coral;">Hello Learners!!</h2>

'
    responseMsg += '<small>Request genrated at: ' + req.requestDate + '</small>'
    res.send(responseMsg)
})

//PORT ENVIRONMENT VARIABLE
const port = process.env.PORT || 8080;
app.listen(port, () => console.log(`Listening on port ${port}..`));
```

3. Cookies

Cookies in Express.js are the small files of information which are stored within a user's computer. They hold a decent amount of data specific to a particular user and the websites he visits. Each time a client loads a website on his browser, the browser will implicitly send the locally stored data back to the website/server, in order to recognize the user. A cookie generally consists of the following:

1. Name
2. Value
3. Zero or more attributes in key-value format. These attributes can include cookie's expiration, domain, and flags, etc.

Now in order to use cookies in express, first you need to install the cookie-parser middleware through ***npm*** into the ***node_modules*** folder that is already present in your project folder. Below is the code to perform the same:

```
npm install cookie-parser
```

Once done, the next step is to import the cookie-parser into your application. Below is code to do the same:

```
var express = require('express');
var cookieParser = require('cookie-parser');
var app = express();
app.use(cookieParser());
```

Now, let me show you how to create and fetch the details of a cookie using cookie-parser.

```
var express = require('express');
var cookie_parser = require('cookie-parser');
var app = express();
app.use(cookie_parser());

//Setting up a Cookie
app.get('/setcookie',function(req, res){
    res.cookie('cookie_name', 'Express_Demo');
    res.cookie('organization', 'Edureka');
    res.cookie('name', 'Swatee');

    res.status(200).send('
<h4 style="font-family: Tahoma; color: coral; text-align: center;">Setting up the Cookie</h4>

');
});

//Checking cookie is set or not
app.get('/getcookie', function(req, res) {
    res.status(200).send(req.cookies);
});

//Welcome Message
app.get('/', function (req, res) {
    res.status(200).send('
<h2 style="font-family: cursive; color: lightseagreen; text-align: center; ">Welcome to Edureka's Expre

');
});

//PORT ENVIRONMENT VARIABLE
const port = process.env.PORT || 8080;
app.listen(port, () => console.log(`Listening on port ${port}..`));
```

You can also check whether the cookie is set in your browser console. For that open up the browser console and type in the following command:

```
console.log(document.cookie);
```

Now that you are aware of how to use middlewares and cookies with Express, in the next section of this Express.js Tutorial lets dive deeper and learn how to create a REST API.

4. REST APIs

REST or RESTful stands for REpresentational State Transfer. It is an architectural style as well as an approach for communications purpose that is often used in various web services development. In simpler terms, it is an application program interface (API) which makes use of the HTTP requests to GET, PUT, POST and DELETE the data over WWW.

Since Express.js is developed on the middleware module of Node.js called connect, it automatically becomes the perfect choice for a server. This is because Express smoothens out the process of creating and exposing APIs for the server to further communicate as a client with the server application. Here the connect module makes use of the http module to communicate with Node.js. Thus, if you are working with any of the connect based middleware modules, then you can easily integrate with Express.js.

If you wish to learn how to create a REST API from scratch you can refer to my article on REST API with Node.js and Express.

Let's now move further in this Express.js Tutorial and understand the concept of scaffolding.

5. Scaffolding

While working with REST APIs you must have seen that you need to work with various files such as HTML, CSS, JPEGs, etc., for accomplishing any particular kind of task. But it becomes too much off of hassle to manage and maintain these files. Thus Express provides the developers an easy solution to this called Scaffolding. Scaffolding is the process of creating a proper structure for a web application and helps in managing the files in proper directories saving us from the manual work.

Thus, you can say that Scaffolding helps in creating the skeleton for our express application and lets us directly jump into the application development. Express provides various Scaffolding tools such as Yeoman and Express-generator. In this Express.js Tutorial, I will show you how to use Express-generator to create the basic structure of an application.

Using express-generator

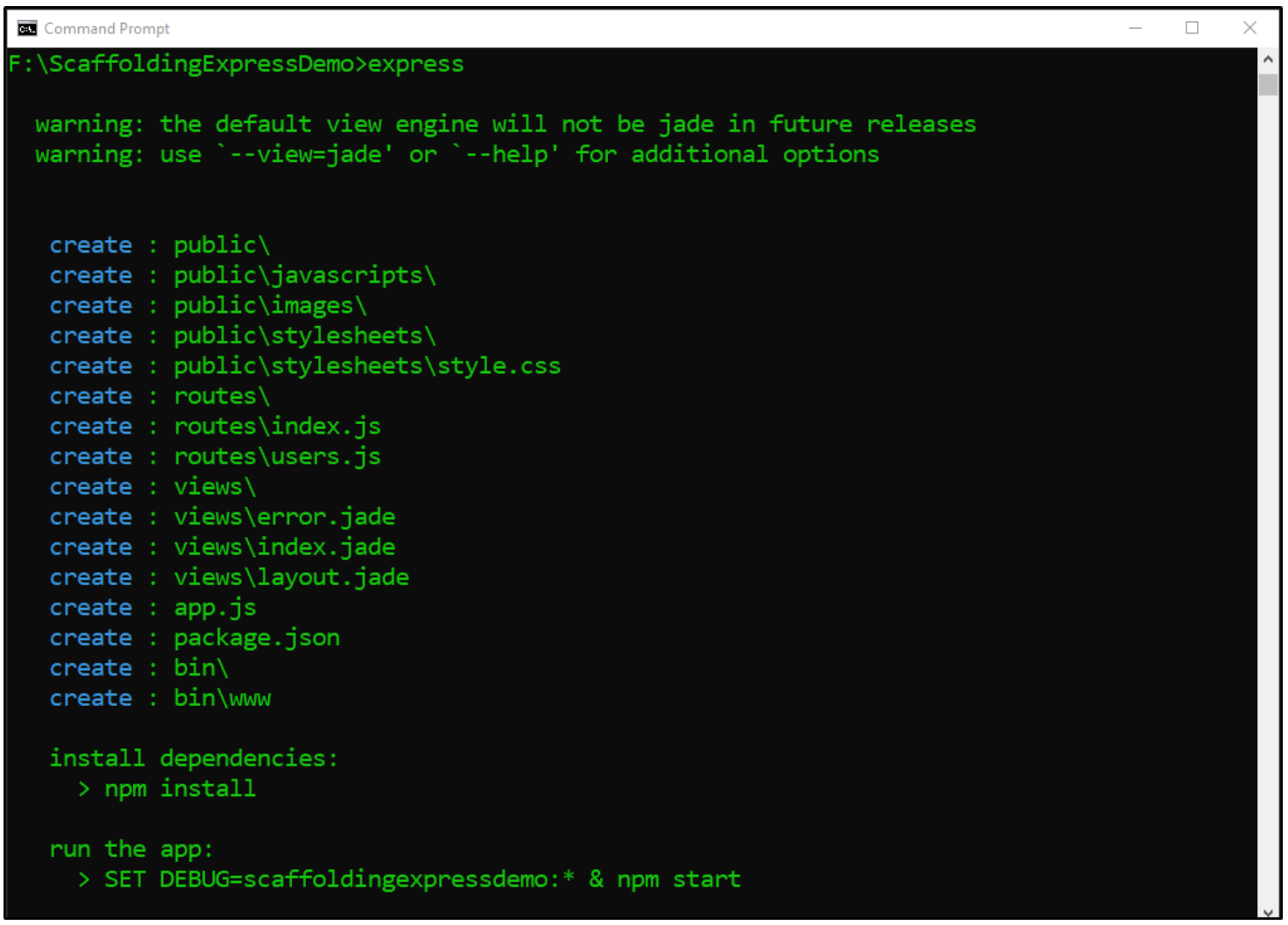
Before you can use express-generator, you need to install this in your system. Type in the below code in the command prompt for the same:

```
npm install -g express-generator
```

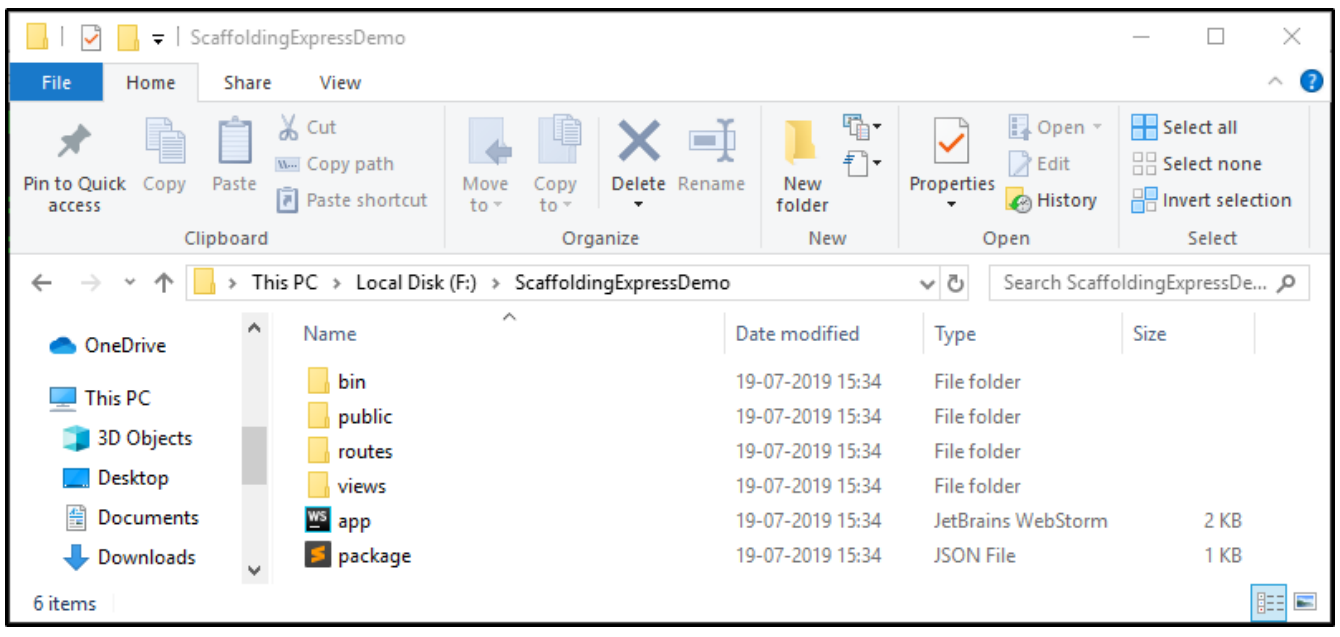
Now you can use express-generator to create the project skeleton. For that you need to type in the following command:

```
express
```

Once you hit enter, your output should look like below screenshot:



If you are getting your output similar to this, that means your skeleton has been created. Now you can go back to your project folder and check the project structure created by scaffolding. Below I have attached the screenshot of my scaffolding.



So that was all about Scaffolding in Express.js. In the next section of this Express.js Tutorial, I will tell you how to establish database connectivity from within an Express application.

6. Database Connectivity

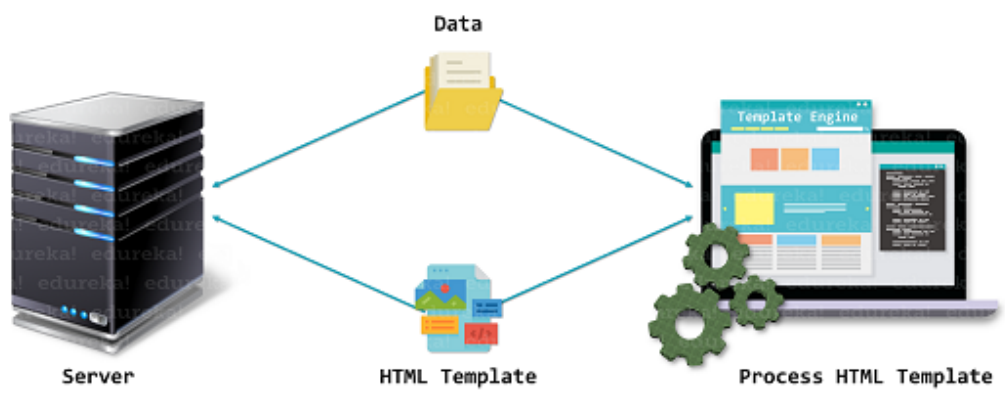
There are a number of databases available in the market out of which most popular ones among front end developers are MySQL and MongoDB. I have written blogs on how to connect with each of these from scratch:

1. Connecting a REST API with **MySQL**
2. Connecting REST API with **MongoDB**

In the next section of this Express.js Tutorial, I will be introducing you to the concept of templating that is prominently used with Express and Node.js applications.

7. Templating

Templating engine facilitates us in creating and using the static HTML template with minimal code. When you execute them, the variables in the template file gets replaced by actual values and the entire template is converted into an HTML file by the template engine. Then, it is finally sent to the client-side where data is further injected into the actual template to produce the final HTML document. This reduces the page load time as well as the designing complexity thus is majorly used for designing the HTML pages.



Below I have listed down some of the popular template engines that work well with Express.js:

- Pug (earlier know as Jade)
- mustache
- dust
- handlebars
- hogan
- liquor
- toffee
- underscore
- walrus
- whiskers

Let's now move further in this Express.js Tutorial and see how to handle exceptions in any Express.js application.

8. Error Handling

Error Handling in Express.js is the mechanism for catching and processing the possible errors which might occur during the program execution and disrupt the normal flow. Express, takes care of errors with the help of a middleware which is further categorized in various types based on the number of arguments the middleware function takes in. Here, I will be talking

specifically about the “error handling middleware” function which takes in four arguments namely, err, req, res and next. This function is invoked only if an error occurs. Below I have given an example for the same:

```
app.use(function (err, req, res, next) {
    console.error(err.stack);
    res.status(500).send('
<h2 style="font-family: Malgun Gothic; color: darkred;">Ooops... Cant find what you are looking for!</h2

');
});
```

With this, we come to an end of this Express.js Tutorial. I hope I was able to explain the concepts of Express.js from the ground up.

*If you found this “Express.js Tutorial” relevant, check out the **Node.js Certification Training** by Edureka, a trusted online learning company with a network of more than 250,000 satisfied learners spread across the globe.*

Got a question for us? Please mention it in the comments section of this Express.js Tutorial and we will get back to you.